

RNN 기반으로 하는 자연어처리

1. 자연어처리란?
2. 데이터셋 소개 - NSMC , KcBERT
3. Word2Vec 과 토큰나이저
4. Recurrent Neural Network
5. RNN 의 한계와 해결법 (LSTM, GRU)

1. 자연어처리란?

1. 자연어처리란?

1.1 자연어(Natural Language)

- 자연어(Natural Language)의 정의

- 자연어는 인간이 일상생활에서 의사소통을 위해 사용하는 언어입니다.
- 사람들 간의 상호작용에 필수적인 도구로, 특정한 규칙(문법)에 기반하여 형성된 복잡한 체계입니다.

모호성(Ambiguity): 단어나 문장은 문맥에 따라 다르게 해석될 수 있습니다.

복잡성(Complexity): 자연어는 문법적으로 매우 복잡하며, 다양한 규칙과 예외가 존재합니다.

다양성(Diversity): 전 세계에는 수천 개의 자연어가 존재하며, 각 언어는 고유한 구조와 표현을 가지고 있습니다.

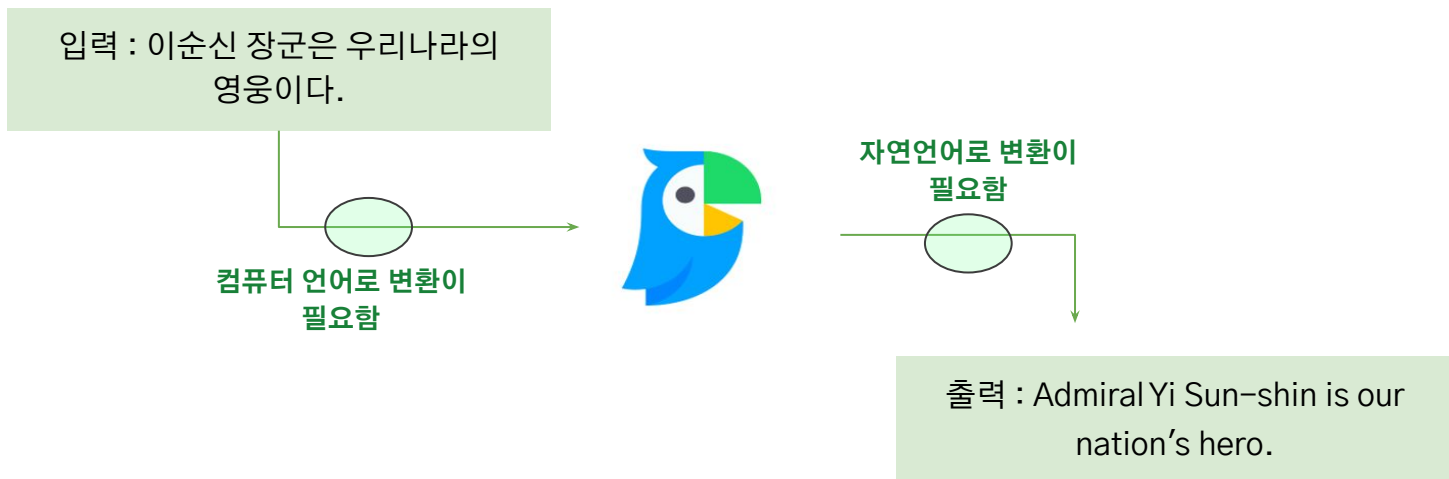
내가 한국어 자연어처리를 한다면, 언어학적인 지식이 있을수록 유리하다.

1. 자연어처리란?

1.1 자연어(Natural Language)

- 자연어와 컴퓨터언어

- 자연어, 사람이 이해할 수 있는 데이터 : 텍스트 형태 e.g) 이순신 장군은 우리나라의 영웅이다.
- 컴퓨터 언어, 컴퓨터가 이해할 수 있는 데이터 : 숫자 형태 e.g) [2, 10661, 2073, 7339, 2079, 7073, 28674, 3]

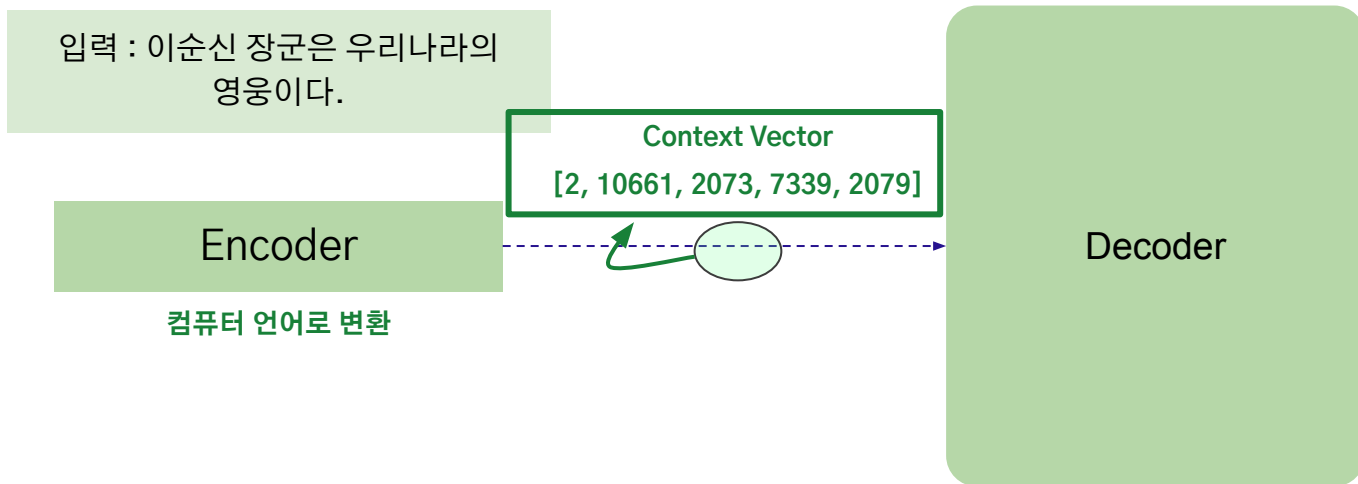


1. 자연어처리란?

1.2 인코더(Encoder)와 디코더(Decoder)

- 인코더(Encoder)

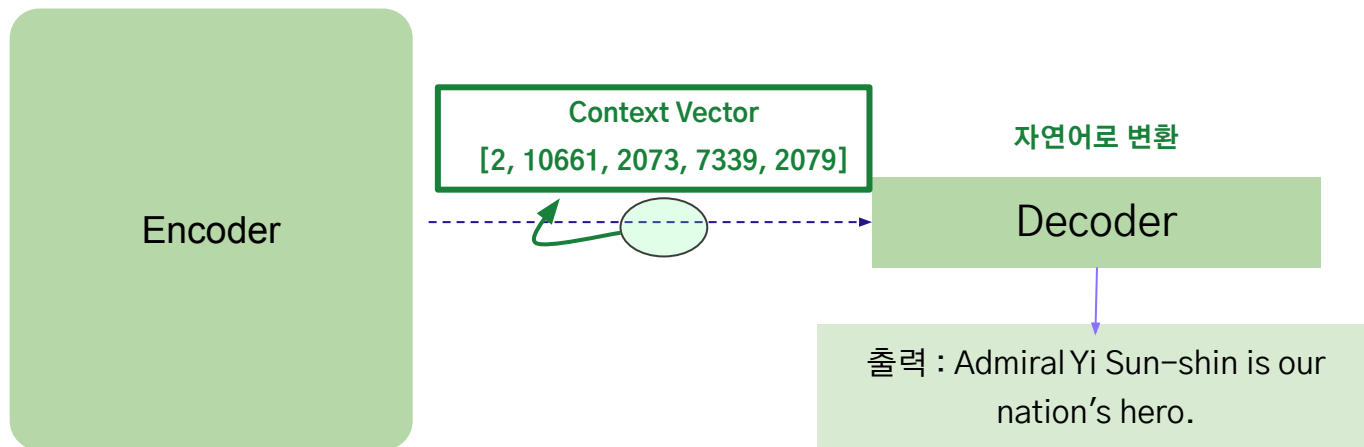
- 사람이 이해할 수 있는 자연어를 컴퓨터가 이해할 수 있는 컴퓨터 언어로 변환해주는 장치
- 아래 그림과 같이, 입력 시퀀스 문장을 고정된 길이의 벡터 (Context vector)로 변환



1. 자연어처리란?

1.2 인코더(Encoder)와 디코더(Decoder)

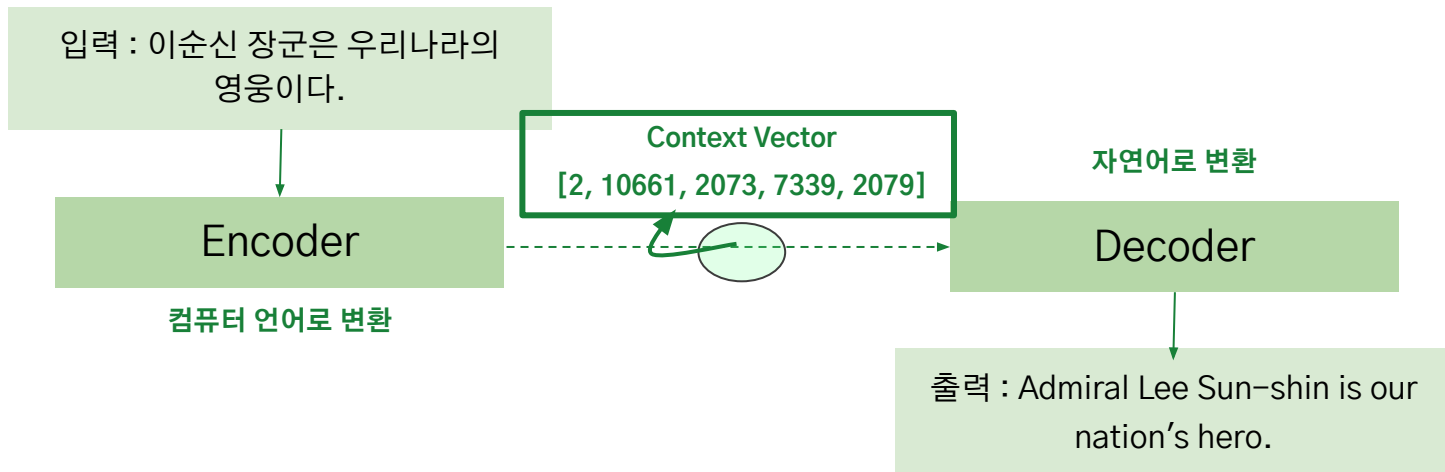
- 디코더(Decoder)
- 컴퓨터가 이해할 수 있는 컴퓨터 언어를 사람이 이해할 수 있는 자연어로 변환해주는 장치
- 아래 그림과 같이, 고정된 길이의 벡터 (Context vector)를 입력으로 받아, 출력 문장으로 변환



1. 자연어처리란?

1.2 인코더(Encoder)와 디코더(Decoder)

- Seq2Seq (Sequence to Sequence) 구조
 - 인코더 (Encoder)와 디코더 (Decoder) 를 결합한 구조
 - 기계번역 기준, 입력 언어를 고정된 길이의 벡터로 변환한 후에, (고정된 길이의 벡터를) 타겟 언어로 번역하는 구조

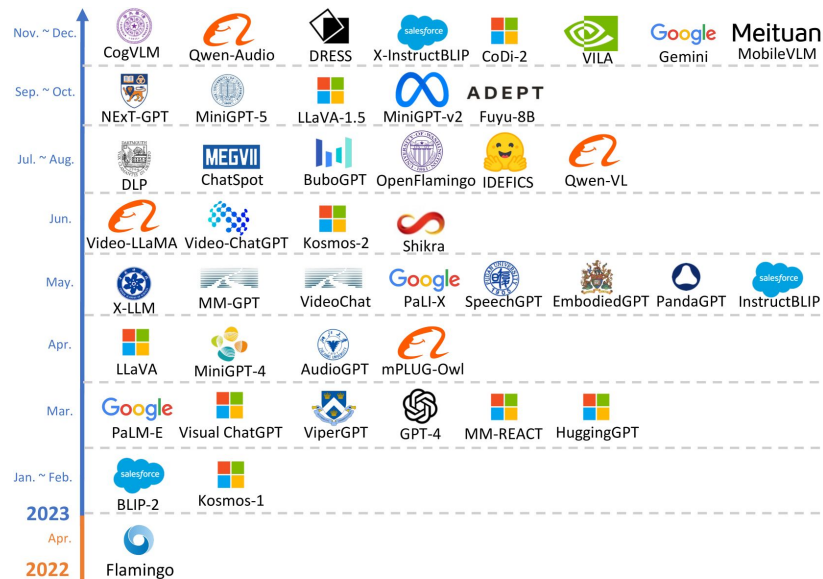


1. 자연어처리란?

1.3 언어모델(Language Model)

• 언어모델이란?

- 언어 모델은 주어진 문맥에서 다음 단어를 예측하거나 문장의 구조와 의미를 이해하여 텍스트를 생성하는 데 사용된다.
- 언어 모델은 텍스트 데이터를 학습하여, 단어들의 확률 분포를 예측합니다.



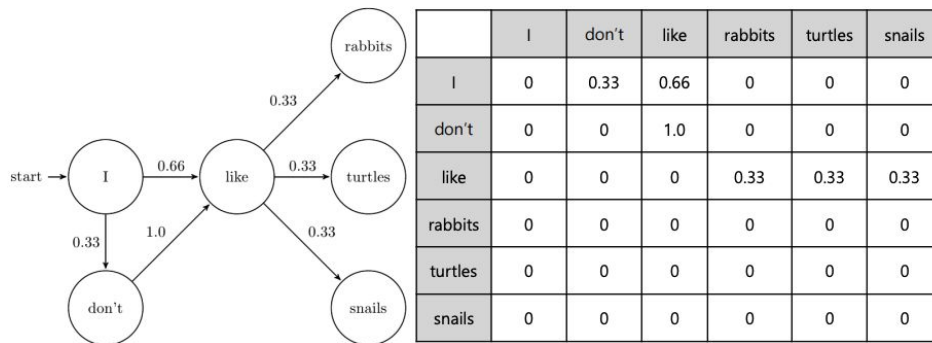
1. 자연어처리란?

1.3 언어모델(Language Model)

• 마코브 체인(Markov Chain) - 초기의 언어모델

- 현재 상태가 주어지면 미래의 상태가 과거 상태와 상관없이 현재 상태에 의해서만 결정된다는 개념을 기반으로 한다.
- 이 모델은 통계적 예측과 의사결정 문제에서 널리 사용되며, 특히 순차적 데이터를 처리할 때 유용합니다.

$$P(X_{n+1} = s_{n+1} | X_n = s_n, X_{n-1} = s_{n-1}, \dots, X_0 = s_0) = P(X_{n+1} = s_{n+1} | X_n = s_n)$$



"I like rabbits"

"I like turtles"

"I don't like snails"

→

→

→

$$0.66 * 0.33 = 0.22$$

$$0.66 * 0.33 = 0.22$$

$$0.33 * 1.0 * 0.33 = 0.11$$

1. 자연어처리란?

1.4 학회를 통해 보는 자연어처리 분야

- 자연어 처리(Natural Language Processing) – 주요 학회
 - **ACL** (Association for Computational Linguistics) : <https://aclanthology.org/>
 - **EMNLP** (Empirical Methods in Natural Language Processing) : <https://2024.emnlp.org/>
 - **NAACL** (North American Chapter of the ACL) : <https://2024.naacl.org/>

1. 자연어처리란?

1.4 학회를 통해 보는 자연어처리 분야

- 자연어 처리(Natural Language Processing) – 학문 분야

1. **Low-level Parsing**
 - Tokenization
 - Stemming / **Lemmatization**
2. **Word and Phrase Level**
 - **NER (Named Entity Recognition)**
 - **Entity Linking**
 - POS (Part-Of-Speech Tagging)
3. **Sentence Level**
 - 감성 분류 (Sentiment Analysis)
 - 기계 번역 (Machine Translation)
4. **Multi-Sentence and Paragraph Level**
 - 논리적 내포 및 모순관계 예측 (Entailment Prediction) / **Natural Language Inference (NLI)**
 - 독해 기반 질의응답 (Question Answering)
 - 챗봇 (Dialog Systems), 요약 (Summarization)

1. 자연어처리란?

1.4 학회를 통해 보는 자연어처리 분야

- 텍스트 마이닝(Text Mining) - 주요 학회
 - KDD (Knowledge Discovery and Data Mining) : <https://www.kdd.org/>
 - The WebConf (World Wide Web Conference) : <https://www2024.thewebconf.org/>
 - WSDM (Web Search and Data Mining) : <https://www.wsdm-conference.org/2024/>
 - CIKM (Conference on Information and Knowledge Management) : <https://cikm2024.org/>
 - ICWSM (International Conference on Web and Social Media) : https://www.icwsml.org/2024/index.html/call_for_submissions.html

1. 자연어처리란?

1.4 학회를 통해 보는 자연어처리 분야

- 텍스트 마이닝(Text Mining) - 학문 분야

1. 텍스트와 문서 데이터로부터 유용한 정보 및 통찰 추출

- a. 주제 발견(Topic Discovery)
- b. 감정 분석(Opinion Mining)

2. 문서 군집화 (Document Clustering)

- 토픽 모델링 (Topic Modeling)
 - 대표 기법: Latent Dirichlet Allocation (LDA), Non-negative Matrix Factorization (NMF)

3. 사회과학적 분석과 연계

- Computational Social Science
- 소셜 미디어 분석 (Social Media Analytics)

1. 자연어처리란?

1.4 학회를 통해 보는 자연어처리 분야

- 정보 검색(Information Retrieval) - 주요 학회
 - **SIGIR** (Special Interest Group on Information Retrieval) : <https://sigir.org/sigir2023/>
 - **WSDM**, **CIKM**, **Recsys** (Recommendation Systems) : <https://recsys.acm.org/recsys23/accepted-contributions/>

1. 자연어처리란?

1.4 학회를 통해 보는 자연어처리 분야

• 정보 검색(Information Retrieval) – 학문 분야

1. 정보 검색 (Information Retrieval, IR)

- 문서 순위 매기기 (Document Ranking) - TF-IDF, BM25, PageRank , BERT
- 자연어 처리(NLP)와 연계: 쿼리의 의도 이해, 단어 간의 관계 파악 등을 통해 검색 정확도를 높이는 연구
- 대규모 데이터 처리: 분산 컴퓨팅을 활용해 실시간으로 데이터를 처리하고 검색 결과를 반환이 중요

2. 추천 시스템 (Recommendation Systems)

- 협업 필터링(Collaborative Filtering): 다른 사용자들의 유사한 행동 패턴을 분석해 추천
- 콘텐츠 기반 필터링(Content-based Filtering): 아이템의 속성(장르, 키워드 등)을 분석해 추천
- **Next-item Prediction**: 사용자가 다음에 상호작용할 아이템을 예측 (전자상거래나 콘텐츠 스트리밍 서비스에서 자주 활용)
- **Heterogeneous Graphs**: 여러 유형의 노드(사용자, 아이템 등)와 다양한 관계를 고려하여 추천

3. Computational Social Science와 관련성

- 소셜 네트워크 분석
- 여론 분석 및 감정 분석

1. 자연어처리란?

1.4 학회를 통해 보는 자연어처리 분야

• 2024년 자연어처리 최신 트렌드

1. Foundation Models 및 사전 학습된 언어 모델(PLM)의 고도화

- **GPT-4, LLaMA, PaLM** 등 대규모 모델이 더 정교해지고 확장된 사용 사례에 적용
- **Multimodal Models**: 텍스트 외에 이미지, 오디오 등 여러 유형의 데이터를 동시에 처리하는 모델 (예: GPT-4, DALL-E)
- **Instruction Tuning**: 사용자의 명령어를 이해하고 이를 기반으로 다양한 작업을 수행하는 모델의 발전

2. Efficient NLP: 모델의 크기와 비용을 줄이는 최적화 기법

- **Parameter Efficient Tuning (PET)**: 전체 모델을 재학습하지 않고, 일부 파라미터만 조정하여 성능 향상
- **Distillation, Pruning, Quantization** 같은 경량화 기법들이 연구되고 상용화
- **LoRA (Low-Rank Adaptation)**: 대규모 모델을 더 작은 컴퓨팅 자원으로 효과적으로 사용하는 방법

3. Natural Language Understanding (NLU)와 Generation (NLG)의 결합

- **Generative AI**: 대규모 언어 모델을 활용한 창의적인 글쓰기, 자동화된 콘텐츠 생성, 코드 작성 등의 고도화
- NLU의 개선을 바탕으로 더 복잡한 의도 해석, 추론 작업 가능

1. 자연어처리란?

1.4 학회를 통해 보는 자연어처리 분야

- 2024년 자연어처리 최신 트렌드

4. Human-in-the-Loop (HITL) 시스템의 강화

- 사람의 피드백을 모델 학습에 통합해 정확성과 신뢰성 향상
- 예: RLHF (Reinforcement Learning with Human Feedback)

5. 언어 모델의 사회적 영향에 대한 연구

- AI 윤리, Bias Mitigation (편향 완화), Fairness 이슈
- Explainability: 모델이 내린 결정의 투명성과 해석 가능성 증대

6. 도메인 특화 모델의 부상

- 금융, 의료, 법률 등의 전문 분야에 맞춘 언어 모델 개발 및 활용
- Knowledge-enhanced Language Models: 지식 그래프나 외부 지식을 모델에 통합해 도메인 지식 처리 강화

7. 실시간 언어 처리 및 상호작용의 발전

- 대규모 언어 모델이 실시간 대화, 음성 인식, 실시간 번역 등에 적용되어 상용화

2. 데이터셋 소개 - NSMC, KcBERT

2. 데이터셋 소개 - NSMC, KcBERT

2.1 Kcbert

1. 데이터셋 설명

- KcBERT 학습에 쓰인 데이터로, **정제되지않은 온라인뉴스의 댓글과 대댓글 수집한 데이터**

2. 다운로드 방법

- Korpora(Korean Corpora Archives) 오픈소스를 통해서 다운로드

```
1 from Korpora import Korpora
2 Korpora.corpus_list()

{'kcbert': 'beomi@github 님이 만드신 KcBERT 학습데이터',
 'korean_chatbot_data': 'songys@github 님이 만드신 챗봇 문답 데이터',
 'korean_hate_speech': '{inmoonlight,warnikchow,beomi}@github 님이 만드신 혐오댓글데이터',
 'korean_parallel_koen_news': 'jungyeul@github 님이 만드신 병렬 말뭉치',
 'korean_petitions': 'lovit@github 님이 만드신 2017.08 ~ 2019.03 청와대 청원데이터',
 'kornli': 'KakaoBrain 에서 제공하는 Natural Language Inference (NLI) 데이터',
 'korsts': 'KakaoBrain 에서 제공하는 Semantic Textual Similarity (STS) 데이터',
 'kowikitext': 'lovit@github 님이 만드신 wikitext 형식의 한국어 위키피디아 데이터',
```

- <https://github.com/ko-nlp/Korpora>
- https://ko-nlp.github.io/Korpora/ko-docs/corpuslist/korean_comments.html
- <https://www.kaggle.com/datasets/junbumlee/kcbert-pretraining-corpus-korean-news-comments>

2. 데이터셋 소개 - NSMC, KcBERT

2.1 Kcbert

Korpora는 오픈소스 말뭉치들의 다운로드와 전처리 기능을 제공하는 파이썬 라이브러리입니다. 복잡한 설명을 피하고 싶으신 분들은 '설치 방법'과 '빠른 사용법'만 읽으셔도 간편하게 사용할 수 있습니다.

1. 데이터셋 설명

- KcBERT 학습에 쓰인 데이터로, 정제되지않은 온라인뉴스의 댓글과 대댓글 수집한 데이터

2. 다운로드 방법

- Korpora(Korean Corpora Archives) 오픈소스를 통해서 다운로드

```
1 from Korpora import Korpora
2 Korpora.corpus_list()
```

```
{'kcbert': 'beomi@github 님이 만드신 KcBERT 학습데이터',
 'korean_chatbot_data': 'songys@github 님이 만드신 챗봇 문단',
 'korean_hate_speech': '{inmoonlight,warnikchow,beomi}@github 님이 만드신 증오피 댓글',
 'korean_parallel_koen_news': 'jungyeul@github 님이 만드신 뉴스',
 'korean_petitions': 'lovit@github 님이 만드신 2017.08 ~ 2018.07 국민청원',
 'kornli': 'KakaoBrain 에서 제공하는 Natural Language Inference',
 'korsts': 'KakaoBrain 에서 제공하는 Semantic Textual Similarity',
 'kowikitext': 'lovit@github 님이 만드신 wikitext 형식의 한국어 위키백과 텍스트'}
```

개돼지 꿀잼이다 이것들이 핵시설 숨겨놓고 모른다고 하면 미국이 가만있었나 빨갱이하는것이 거짓말이 기네 트럼프가 문어병쩍쩍무늬재앙당어리는 아니자나...

기름과 물인데 무엇을 기대하는가 6,25때 미군이 몇명이나 북한군에게 죽었는데 트럼프가 과연 북한을 정정오니는 99% 마음에 들고 1%가 마음에 안들면 안한다. 우리도 명심해야 된다.

이것이 북한의 현실인 거예요

매국노들 북미 회담 결렬되니까 아주 신나가지고 날뛰다

김정은 젓 되 뿌리쏘

원 위상! 즈그집 왕강아지인데

오늘 술 한잔하자 에헤라디아

트럼프는 집사가 청문회에서 발언한것 땀에 정신은 판곳에 가있다...

개정은이 어찌나??

지금 쿠데카각인데??

- <https://github.com/ko-nlp/Korpora>
- https://ko-nlp.github.io/Korpora/en-docs/corpuslist/korean_comments.html
- <https://www.kaggle.com/datasets/junbumlee/kcber-pretraining-corpus-korean-news-comments>

2. 데이터셋 소개 - NSMC, KcBERT

2.1 Kcbert

3. 데이터셋 살펴보기

- 두 가지 방법으로 말뭉치(Corpus) 다운로드 할 수 있다. (시간이 조금 걸린다.)

```
1 from Korpora import Korpora
2 Korpora.corpus_list()
3 corpus = Korpora.load("kcbert")

1 from Korpora import KcBERTKorpus
2
3 corpus = KcBERTKorpus()
```

Korpora 는 다른 분들이 연구 목적으로 공유해주신 말뭉치들을 손쉽게 다운로드, 사용할 수 있는 기능만을 제공합니다.

말뭉치들을 공유해 주신 분들에게 감사드리며, 각 말뭉치 별 설명과 라이선스를 공유 드립니다.
해당 말뭉치에 대해 자세히 알고 싶으신 분은 아래의 description 을 참고,
해당 말뭉치를 연구/상용의 목적으로 이용하실 때에는 아래의 라이선스를 참고해 주시기 바랍니다.

```
# Description
Author : beomi@github
Repository : https://github.com/Beomi/KcBERT/
References :
```

공개된 한국어 BERT는 대부분 한국어 위키, 뉴스 기사, 책 등 잘 정제된 데이터를 기반으로 학습한 모델입니다.

한편, 실제로 NSMC와 같은 댓글형 데이터셋은 정제되지 않았고 구어체 특징에 신조어가 많으며, 오타자 등 공식적인 글쓰기에서 나타나지 않는 표현들이 빈번하게 등장합니다.

KcBERT는 위와 같은 특성의 데이터셋에 적용하기 위해, 네이버 뉴스에서 댓글과 대댓글을 수집해, 토큰라이저와 BERT모델을 처음부터 학습한 Pretrained BERT 모델입니다.

KcBERT는 Huggingface의 Transformers 라이브러리를 통해 간편히 불러와 사용할 수 있습니다.
(별도의 파일 다운로드가 필요하지 않습니다.)

```
# License
MIT License
```

```
[kcbert] download kcbert-train.tar.gzaa: 100%|#####| 2.10G/2.10G [00:38<00:00, 54.0MB/s]
[kcbert] download kcbert-train.tar.gzab: 100%|#####| 2.10G/2.10G [00:40<00:00, 52.2MB/s]
[kcbert] download kcbert-train.tar.gzac: 671MB [00:14, 46.2MB/s]
Unzip tar. It needs a few minutes ...
```

- <https://github.com/ko-nlp/Korpora>
- https://ko-nlp.github.io/Korpora/en-docs/corpuslist/korean_comments.html
- <https://www.kaggle.com/datasets/iunbumlee/kcbert-pretraining-corpus-korean-news-comments>

2. 데이터셋 소개 - NSMC, KcBERT

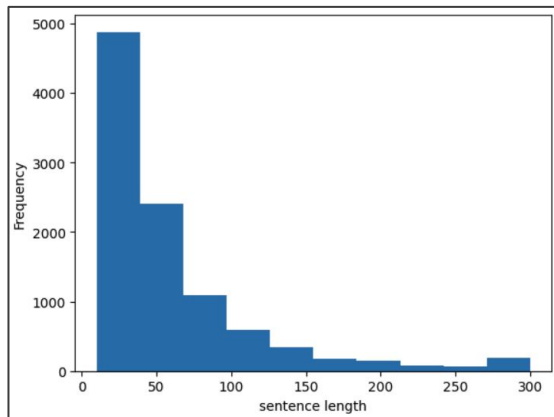
2.1 Kcbert

3. 데이터셋 살펴보기

- 전체 불러오는건 RAM 이슈가 있으므로 지정된 갯수로도 불러올 수 있다.
- 간단하게 문장 길이 분포 확인하면 아래와 같다.

```
1 len(corpus.train[:])  
  
10000
```

```
제일 긴 문장길이 : 300  
제일 짧은 문장길이 : 10  
평균 문장길이 : 58.8614
```



2. 데이터셋 소개 - NSMC, KcBERT

2.2 NSMC

1. 데이터셋 설명

- Naver Movie Sentiment data, 한국어 영화 리뷰 데이터 20만개의 리뷰와 긍부정 레이블로 구성되어있다.

2. 다운로드 방법

- 지정된 URL 내에서 데이터셋 다운로드

```
1 urllib.request.urlretrieve("https://raw.githubusercontent.com/e9t/nsmc/master/ratings_train.txt", filename="train.txt")
2 urllib.request.urlretrieve("https://raw.githubusercontent.com/e9t/nsmc/master/ratings_test.txt", filename="test.txt")

('test.txt', <http.client.HTTPMessage at 0x7f6dcb3f3a30>)

1 train_data = pd.read_table('train.txt')
2 test_data = pd.read_table('test.txt')
```

	id	document	label
0	9976970	아 더빙.. 진짜 짜증나네요 목소리	0
1	3819312	흠...포스터보고 초딩영화줄....오버연기조차 가볍지 않구나	1
2	10265843	너무재밌었다그래서보는것을추천한다	0
3	9045019	교도소 이야기구먼 ..솔직히 재미는 없다.평점 조정	0
4	6483659	사이폰페그의 익살스런 연기가 돋보였던 영화스파이더맨에서 늙어보이기만 했던 커스틴 ...	1

2. 데이터셋 소개 - NSMC, KcBERT

2.2 NSMC

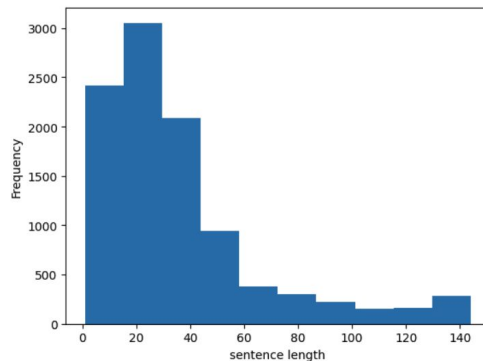
3. 데이터셋 살펴보기

- 간단하게 데이터 갯수, 길이 살펴보기

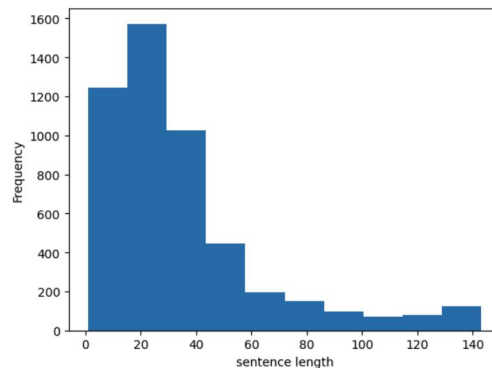
```
1 len(train_data)
150000

1 len(test_data)
50000
```

```
train 중 제일 긴 문장길이 : 144
train 중 제일 짧은 문장길이 : 1
train 평균 문장길이 : 35.8355
```



```
test 중 제일 긴 문장길이 : 143
test 중 제일 짧은 문장길이 : 1
test 평균 문장길이 : 34.5232
```



3. Word2Vec 과 토크나이저

3. Word2Vec 과 토큰나이저

3.1 필요한 기초지식

- 워드 임베딩(Word Embedding)이란?

단어를 밀집 벡터(dense vector)로 표현하는 방법

여기서 밀집 벡터에 반대되는 개념이 희소 벡터(sparse vector)이다.

- 우리가 살펴봤던 원핫인코딩된 벡터같은 것들이 희소 벡터

Sparse Vector 예시 : $[0, 0, 0 \cdots 1, \cdots 0]$, dim 10,000

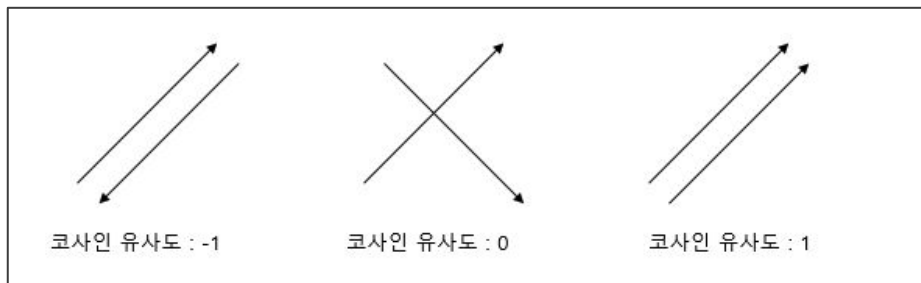
Dense Vector 예시 : $[3, 2, 5, 6, 8, 1, 2, \cdots]$, dim 128

3. Word2Vec 과 토큰나이저

3.1 필요한 기초지식

- 코사인 유사도란?

아래와 같이 벡터들 간의 방향 각도 차이에 따라 계산된 유사도 계산 방식



```
def cos_sim(A, B):  
    return dot(A, B)/(norm(A)*norm(B))
```

$$similarity = \cos(\Theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n (A_i)^2} \times \sqrt{\sum_{i=1}^n (B_i)^2}}$$

3. Word2Vec 과 토큰나이저

3.1 필요한 기초지식

- 코사인 유사도를 이용하여 문서간 유사도 구하기

단어의 빈도수를 기반으로 문장 -> 벡터로 변환 (워드 임베딩) Bag of Words(BoW)

문서1: 저는 사과 좋아요
문서2: 저는 바나나 좋아요
문서3: 저는 바나나 좋아요 저는 바나나 좋아요



	바나나	사과	저는	좋아요
문서1	0	1	1	1
문서2	1	0	1	1
문서3	2	0	2	2



doc1 = [0,1,1,1]
doc2 = [1,0,1,1]
doc3 = [2,0,2,2]

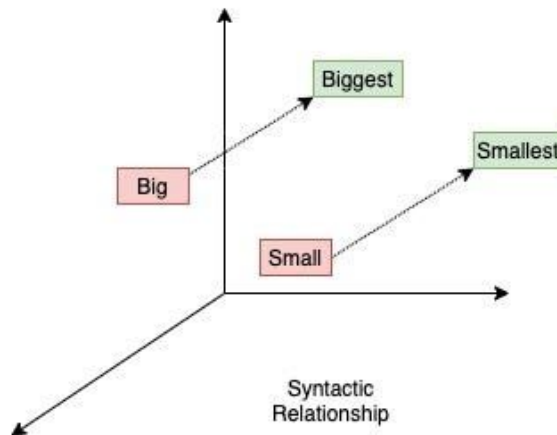
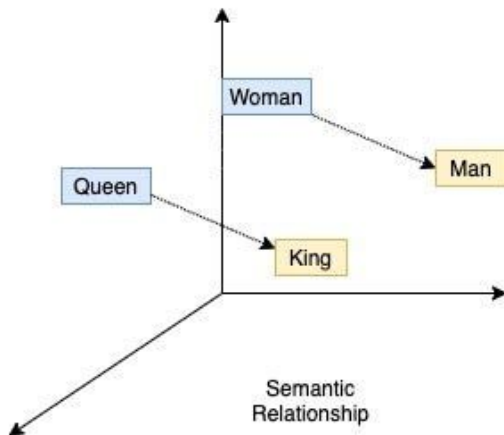
```
print('문서 1과 문서2의 유사도 : ',cos_sim(doc1, doc2))  
print('문서 1과 문서3의 유사도 : ',cos_sim(doc1, doc3))  
print('문서 2와 문서3의 유사도 : ',cos_sim(doc2, doc3))
```

문서 1과 문서2의 유사도 : 0.67
문서 1과 문서3의 유사도 : 0.67
문서 2과 문서3의 유사도 : 1.00

3. Word2Vec 과 토큰나이저

3.2 워드투벡터(Word2Vec)

- 2013년 토론토 대학에서 발표한 워드 임베딩 기법(word를 vector로 바꾸는 기법)
- 신경망 모델을 사용하여 방대한 말뭉치(Corpus)에서 단어의 연관성을 학습한다.
- skip-gram 모델과 , CBOW 모델 2가지가 있다.



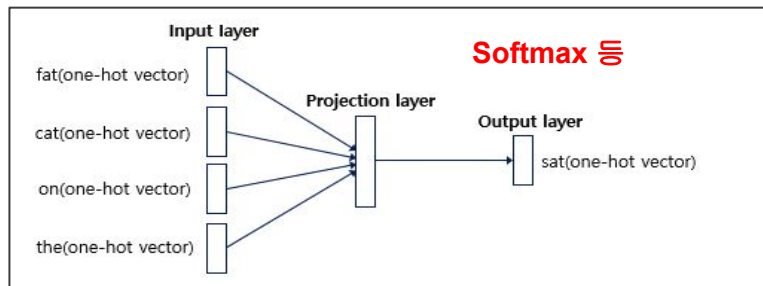
3. Word2Vec 과 토큰나이저

3.2 워드투벡터(Word2Vec)

- CBOW(Continuous Bag of Words)

- CBOW는 주어진 문맥(Context)으로부터 타깃 단어를 예측하는 방식의 워드 임베딩(Word Embedding) 기법
- 입력(Input): 문맥 단어들 (Context Words)
- 출력(Output): 중심 단어(Target Word)

중심 단어	주변 단어
The fat cat sat on the mat	[1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0]
The fat cat sat on the mat	[0, 1, 0, 0, 0, 0, 0], [1, 0, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0]
The fat cat sat on the mat	[0, 0, 1, 0, 0, 0, 0], [1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0]
The fat cat sat on the mat	[0, 0, 0, 1, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0]
The fat cat sat on the mat	[0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 1, 0, 0]
The fat cat sat on the mat	[0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 1, 0]
The fat cat sat on the mat	[0, 0, 0, 0, 0, 0, 1], [0, 0, 0, 0, 0, 0, 1], [0, 0, 0, 0, 0, 0, 1]



여러 단어들의 평균을 타깃(목표)로 예측하기
때문에 연산이 간단하고 빠르다.

3. Word2Vec 과 토큰나이저

3.2 워드투벡터(Word2Vec)

- CBOW(Continuous Bag of Words)

- 예시 문장 : "The quick brown fox jumps over the lazy dog"
- 타깃 단어: "fox"
- 문맥 단어: "The", "quick", "brown", "jumps", "over"

[실행순서]

- 문맥 단어 ("The", "quick", "brown", "jumps", "over")를 입력으로 사용합니다.
- 각 단어의 임베딩 벡터를 구한 후, 이 벡터들을 평균화합니다.
- 평균 벡터를 기반으로 타깃 단어 "fox"를 예측합니다.

	중심 단어	주변 단어
	중심 단어	주변 단어
중심 단어 ↓ 주변 단어		
The fat cat sat on the mat	[1, 0, 0, 0, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0]
The fat cat sat on the mat	[0, 1, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0]
The fat cat sat on the mat	[0, 0, 1, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
The fat cat sat on the mat	[0, 0, 0, 1, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
The fat cat sat on the mat	[0, 0, 0, 0, 1, 0, 0]	[0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]
The fat cat sat on the mat	[0, 0, 0, 0, 0, 1, 0]	[0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 0, 1]
The fat cat sat on the mat	[0, 0, 0, 0, 0, 0, 1]	[0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 0, 1]

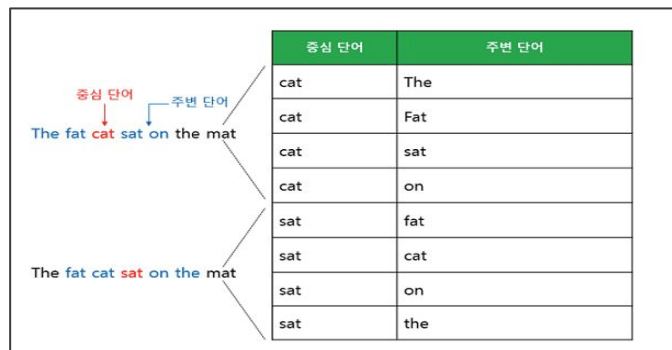
여러 단어들의 평균을 타깃(목표)로 예측하기
때문에 연산이 간단하고 빠르다.

3. Word2Vec 과 토큰나이저

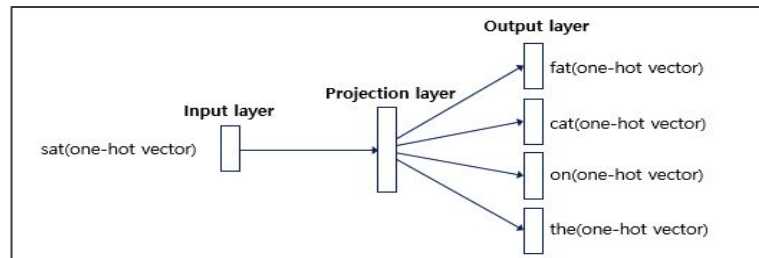
3.2 워드투벡터(Word2Vec)

- Skip-gram

- **Skip-gram**은 주어진 타겟 단어로부터 문맥(주변) 단어들을 예측하는 방식의 워드 임베딩 (Word Embedding) 기법
- **입력(Input):** 중심 단어(Target Word)
- **출력(Output):** 문맥 단어들 (Context Words)



Softmax 또는 Negative Sampling



희귀 단어에 대해서 더 나은 학습 성능을 보인다.

3. Word2Vec 과 토큰나이저

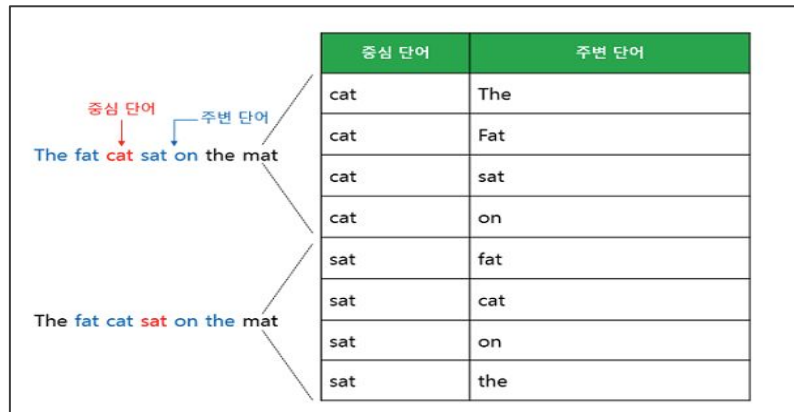
3.2 워드투벡터(Word2Vec)

- Skip-gram

- 예시 문장 : "The quick brown fox jumps over the lazy dog"
- 타깃 단어: "fox"
- 예측해야할 문맥 단어들: "The", "quick", "brown", "jumps", "over"

[실행순서]

- 중심 단어 "*"fox*"를 입력으로 사용합니다.
- 모델은 중심 단어를 바탕으로 주변 단어들 ("The", "quick", "brown", "jumps", "over")을 각각 예측합니다.
- 각 문맥 단어에 대해 예측된 확률 분포를 생성합니다.



희귀 단어나 복잡한 문맥에 대해서 더 나은 학습 성능을 보인다.

3. Word2Vec 과 토큰나이저

3.2 워드투벡터(Word2Vec)

- Word2Vec 예시

1. 도쿄는 일본의 수도, 그렇다면 서울은 어느나라의 수도일까?

일본-도쿄+서울

QUERY

+일본/Noun +서울/Noun -도쿄/Noun

RESULT

한국/Noun

2. 야구하면 박찬호, 축구하면?

박찬호-야구+축구

QUERY

+박찬호/Noun +축구/Noun -야구/Noun

RESULT

호나우두/Noun

3. Word2Vec 과 토큰나이저

3.3 워드투벡터(Word2Vec) 사용해보기

- 사전학습된 Word2Vec

1. 사전학습된 “glove-twitter-25” 를 사용한 유사도

```
1 glove_vectors.similarity('dog', 'cat')  
  
0.95908207
```

2. 두개의 문장으로만 학습한 Word2Vec 모델을 사용한 유사도

```
1 model.wv.similarity('dog', 'cat')  
  
0.23249888
```

3. Word2Vec 과 토큰나이저

3.4 토큰나이저(Tokenizer)

토큰나이저 역할

- 문장 텍스트를 토큰으로 나누는 도구
- 토큰의 기준은 단어, 구, 문자열 등이 될 수 있다.
- 토큰을 나누는 기준에 따라서 여러 종류의 토큰나이저가 있다.

3. Word2Vec 과 토큰나이저

3.4 토큰나이저(Tokenizer)

토큰나이저 동작과정

- 단계 1: 텍스트를 단어 단위로 분리 (간단하게 공백 기준)
- 단계 2: 토큰별로 고유한 인덱스를 부여하여 어휘 사전(vocab) 구축
- 단계 3: 새로운 텍스트가 들어오면 vocab 기준으로 토큰을 인덱스로 변환

“아버지가 방에 들어가신다.”



["아버지가", "방에", "들어가신다", "."]



[32, 5, 135, 3]

3. Word2Vec 과 토큰나이저

3.4 토큰나이저(Tokenizer)

형태소 분석기 - OKt

세종 말뭉치(corpus) 기반으로 **정해진 규칙**대로 토큰나이징한다.
품사를 같이 출력하기에 명사, 동사만 출력가능함.

```
1 from konlpy.tag import Okt
2
3 okt = Okt()
4
5 sentence = "나는 한국어를 공부합니다."
6
7 tokens = okt.morphs(sentence, stem=True)
8
9 print(tokens)
```

['나', '는', '한국어', '를', '공부', '하다', '.']

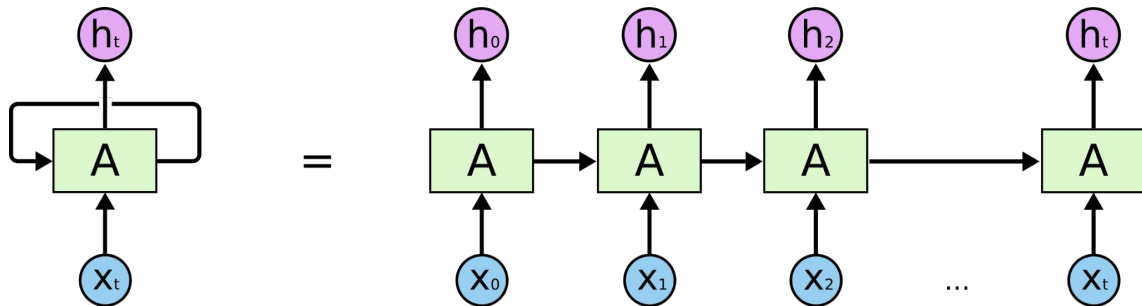
4. Recurrent Neural Network

4. Recurrent Neural Network

4.1 Recurrent Neural Network

- RNN 기본구조

- 재귀적으로 계산되는 원리의 모델 (이전 step의 정보를 이용해 다음 step 계산)
- 이전 스텝($t-1$)의 활성화 함수를 통해 나온 결과값을 출력층 뿐 아니라, 다음 스텝(t)으로 전달하여 활용

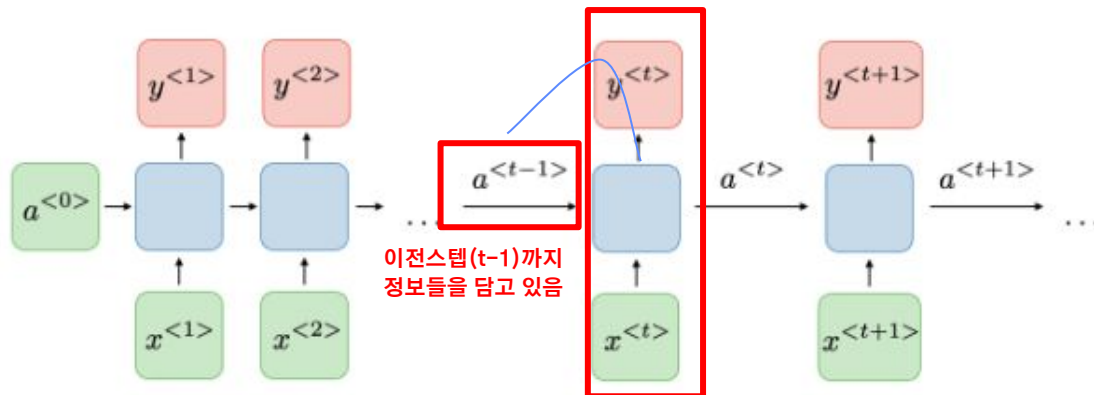


4. Recurrent Neural Network

4.1 Recurrent Neural Network

- RNN 기본구조

- 재귀적으로 계산되는 원리의 모델 (이전 step의 정보를 이용해 다음 step 계산)
- 이전 스텝($t-1$)의 활성화 함수를 통해 나온 결과값을 출력층 뿐 아니라, 다음 스텝(t)으로 전달하여 활용

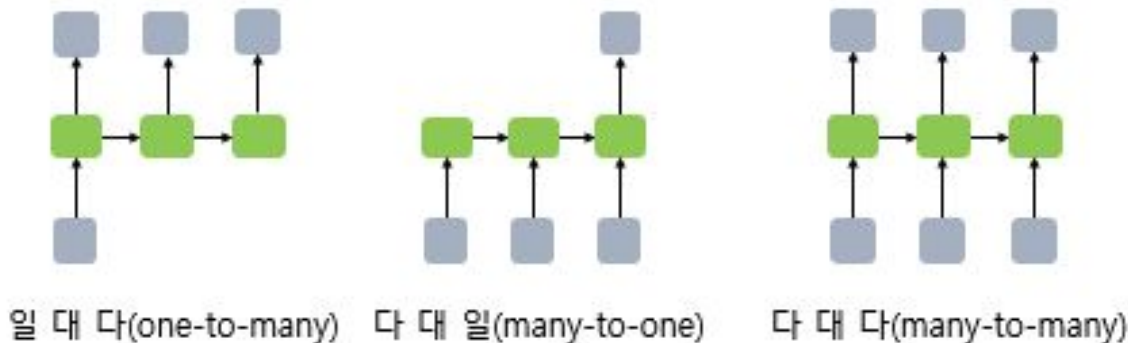


4. Recurrent Neural Network

4.1 Recurrent Neural Network

- RNN 다양한 구조

- 입력값과 출력값이 각각 여러개이거나 한개일 수 있다.

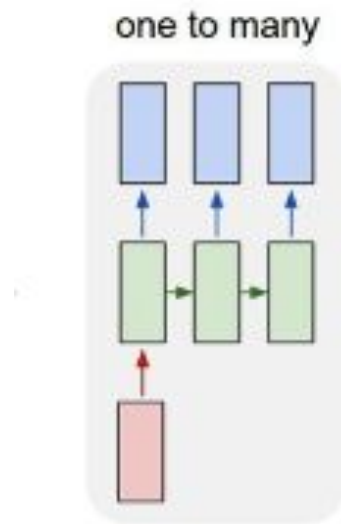


4. Recurrent Neural Network

4.2 RNN 의 사용예시

- 이미지 캡션달기

- 입력된 사진에 해당되는 단어들을 붙여준다.
- 입력 : 사진 픽셀 -> 출력 : 단어들



4. Recurrent Neural Network

4.2 RNN 의 사용예시

- 스팸 이메일 분류

- 메일 내용을 입력받아 스팸여부를 판별한다. (이진분류)
- 입력 : 이메일 텍스트 -> 출력 : 스팸여부(TF)

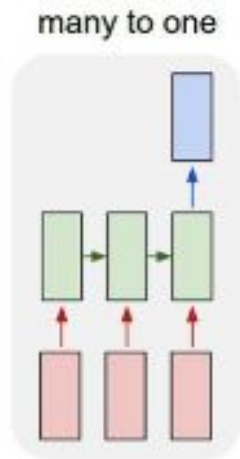


4. Recurrent Neural Network

4.2 RNN 의 사용예시

- 감성 분석

- 입력된 텍스트 기반으로 긍정/부정을 판별한다. (이진분류)
- 입력 : 단어들 -> 출력 : 감성점수(score) 기반 긍정 여부

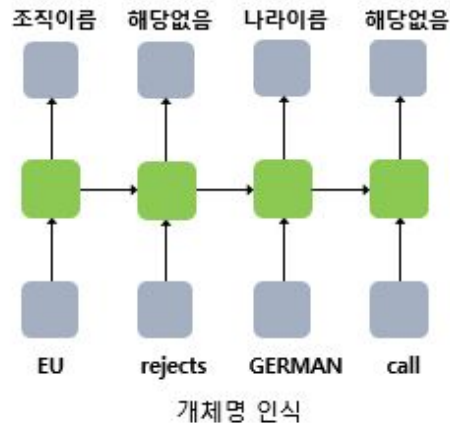


4. Recurrent Neural Network

4.2 RNN 의 사용예시

- 개체명 인식(Named Entity Recognition)

- 각각의 입력값(input)에 대한 개체명(조직이름, 나라이름 품사 등)을 출력해준다.
- 입력 : 문장 내 단어들 -> 출력 : 각 단어별 개체명

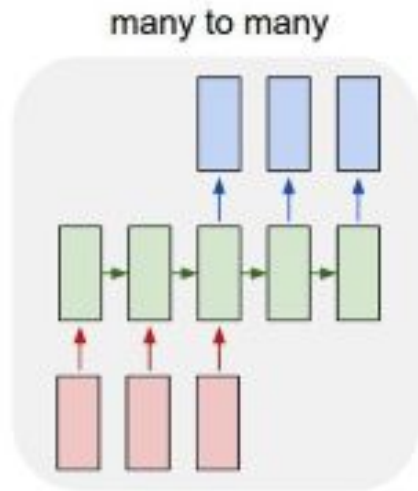


4. Recurrent Neural Network

4.2 RNN 의 사용예시

- 다국어 번역

- 입력된 언어에 대해서 타겟 언어(번역되어야 할 언어)로 출력하며, 입력과 출력의 "시차"가 있다.
- 입력 : 원문 언어 -> 출력 : 번역 언어

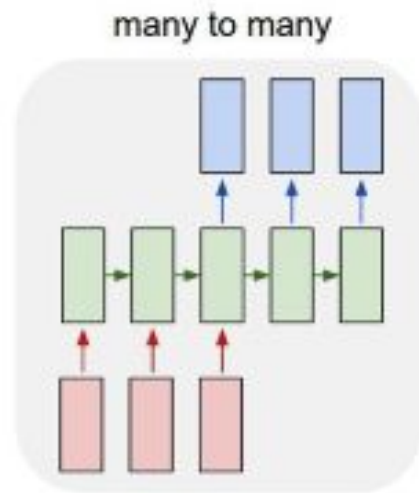


4. Recurrent Neural Network

4.2 RNN 의 사용예시

- 다국어 번역

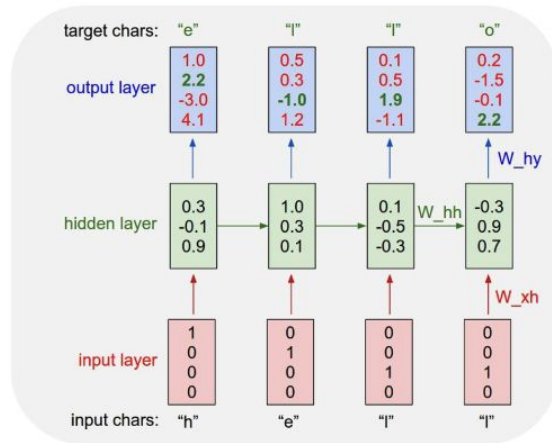
- 입력된 언어에 대해서 타겟 언어(번역되어야할 언어)로 출력하며, 입력과 출력의 "시차"가 있다.
- 입력 : 원문 언어 -> 출력 : 번역 언어



4. Recurrent Neural Network

4.3 RNN 의 구조

- **Input layer** : 모델에 들어가는 텍스트를 워드 임베딩한 값을 받는다.
- **hidden layer** : 입력을 받아, 결과값을 output layer와 다음 스텝의 hidden layer로 전달한다.
- **output layer** : 히든 레이어의 결과값을 받아 출력값을 산출한다.



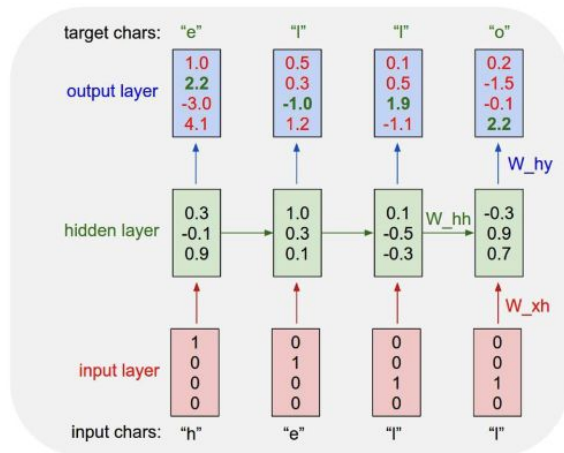
$h = [1, 0, 0, 0]$
 $e = [0, 1, 0, 0]$
 $l = [0, 0, 1, 0]$
 $o = [0, 0, 0, 1]$

4. Recurrent Neural Network

4.3 RNN 의 구조

- RNN 입출력 및 수식

- RNN은 시퀀셜 데이터 (시계열 데이터) 처리를 위해 설계된 모델이므로, 각각의 시간 단계 t 에서의 계산은 이전 시간 단계 $t-1$ 에서의 상태를 활용해 입력을 처리하는 방식이다.
- RNN에서 중요한 부분은 이전 상태 h_{t-1} 과 현재 입력 x_t 를 합쳐서 새로운 상태 h_t 를 계산하는 것.

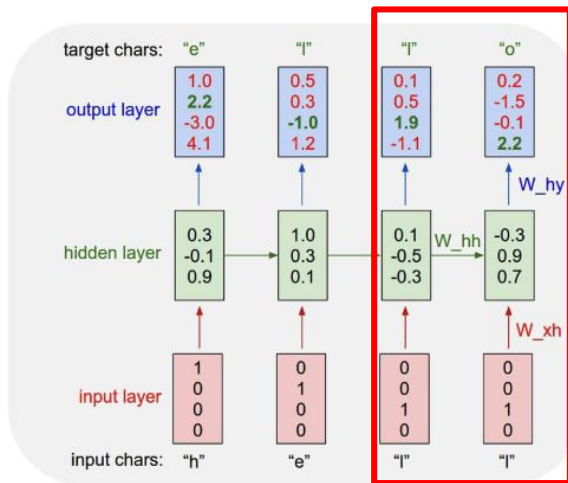


4. Recurrent Neural Network

4.3 RNN 의 구조

- RNN 입출력 및 수식

- RNN의 hidden state :



$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

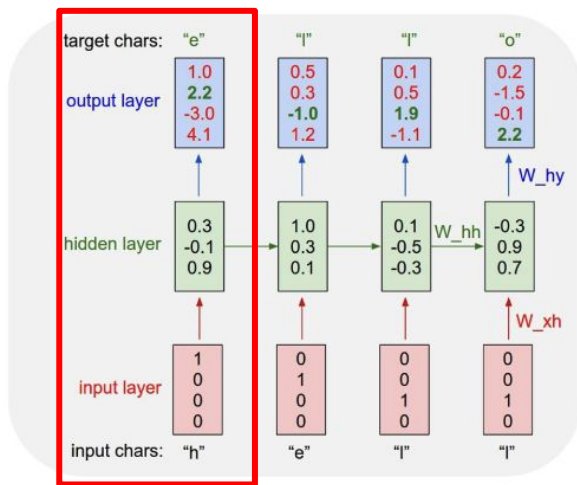
- h_t 는 시간 t 에서의 hidden state.
- h_{t-1} 는 이전 시간의 hidden state.
- x_t 는 시간 t 에서의 입력.
- W_{hh} 는 hidden state에서 hidden state로 가는 가중치 행렬.
- W_{xh} 는 입력에서 hidden state로 가는 가중치 행렬.
- b_h 는 bias term.
- f 는 활성화 함수 (보통 $\tanh$ 또는 $ReLU$).

4. Recurrent Neural Network

4.3 RNN 의 구조

- RNN 입출력 및 수식

2. RNN의 출력(output) :



$$y_t = g(W_{hy}h_t + b_y)$$

- y_t 는 출력값.
- W_{hy} 는 hidden state에서 출력으로 가는 가중치 행렬.
- g 는 출력 활성화 함수 (보통 softmax나 tanh).

4. Recurrent Neural Network

4.4 RNN 의 장단점

장점

- 시퀀스 (sequence) 데이터의 **시퀀셜한 (순서대로 나열되는) 특성**을 학습
- 타임스텝마다 이전 매개변수를 공유하기 때문에 **지난 정보들을 고려하여** 계산
- 입출력 시퀀스의 길이가 다른 것이 가능하며, **다양한 길이의 입출력을 처리하는데 용이함**

단점

- 시퀀스의 길이가 길어질수록 과거의 정보들을 잘 기억하지 못하는 **장기 의존성 문제 (Long-term dependency)**가 존재
- 시퀀스의 **앞선 타임스텝들 계산에 대한 종속성**으로 인하여 계산 속도가 느림
- 학습과정에서 **그레디언트 소실 문제 (Gradient Vanishing Problem)**가 발생

5. RNN의 한계와 해결법(LSTM, GRU)

5. RNN 의 한계와 해결법 (LSTM, GRU)

5.1 RNN의 한계

- **그레디언트 배니싱(Gradient Vanishing) 문제**

- 연쇄법칙(chain rule)에 의해 속미분된 계수 값이 계속 곱해진다.
- 수식 예시 : $f'(3 * g'(3 * h'(3 * k'(x)))) \dots$

- **Vanilla RNN**에서 t 번째 시점의 hidden state는 다음과 같이 계산됩니다:

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_x x^{(t)} + b)$$

- **Chain Rule**을 사용하여, RNN의 역전파 과정에서 각 시점의 그레디언트를 계산합니다.

네 번째 시점의 손실 $J^{(4)}$ 에 대한 첫 번째 시점의 hidden state $h^{(1)}$ 의 Gradient:

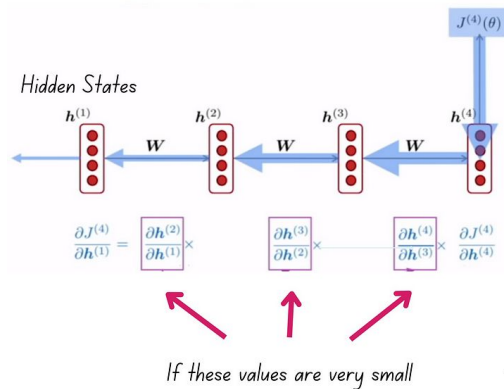
$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

이 과정을 일반화하면, 첫 번째 시점의 손실 $J^{(t)}$ 에 대한 $h^{(1)}$ 의 Gradient는 다음과 같이 표현됩니다:

$$\frac{\partial J^{(t)}}{\partial h^{(1)}} = \frac{\partial J^{(t)}}{\partial h^{(t)}} \prod_{i < t} \frac{\partial h^{(i+1)}}{\partial h^{(i)}}$$

$J^{(t)}$ 를 $h^{(t)}$ 로 미분했을때의 속미분 계수값들의 곱

Back Propagation through Time (BPTT)



Sagar Patil

5. RNN의 한계와 해결법 (LSTM, GRU)

5.1 RNN의 한계

- **그레디언트 배니싱(Gradient Vanishing) 문제**

- 연쇄법칙(chain rule)에 의해 속미분된 계수 값이 계속 곱해진다.
- 수식 예시 : $f'(0.3 * g'(0.3 * h'(0.3 * k'(x)))) \dots$

- Vanilla RNN에서 t 번째 시점의 hidden state는 다음과 같이 계산됩니다:

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_x x^{(t)} + b)$$

- Chain Rule을 사용하여, RNN의 역전파 과정에서 각 시점의 그레디언트를 계산합니다.

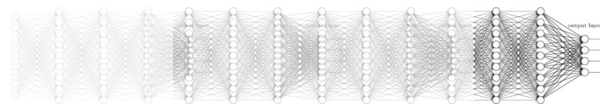
네 번째 시점의 손실 $J^{(4)}$ 에 대한 첫 번째 시점의 hidden state $h^{(1)}$ 의 Gradient:

$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

이 과정을 일반화하면, 첫 번째 시점의 손실 $J^{(t)}$ 에 대한 $h^{(1)}$ 의 Gradient는 다음과 같이 표현됩니다:

$$\frac{\partial J^{(t)}}{\partial h^{(1)}} = \frac{\partial J^{(t)}}{\partial h^{(t)}} \prod_{i=1}^{t-1} \frac{\partial h^{(i+1)}}{\partial h^{(i)}}$$

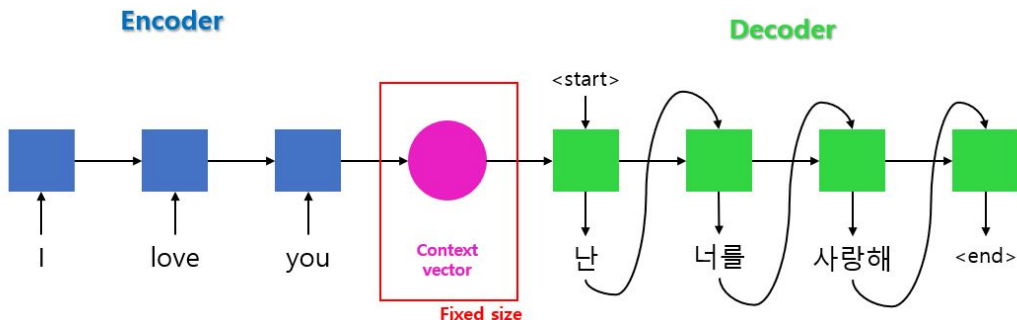
Vanishing gradient (NN winter2: 1986-2006)



5. RNN 의 한계와 해결법 (LSTM, GRU)

5.1 RNN의 한계

- 병목(Bottle neck) 현상
 - 한정된 크기의 Context Vector 를 통해 Encoder 의 내용을 Decoder 에 전달함.



5. RNN 의 한계와 해결법 (LSTM, GRU)

5.1 RNN의 한계

- 병목(Bottle neck) 현상

- Encoder 측 : ㅎㅎ 100가지 이야기를 해놓고 1가지로 요약해 드릴게요.

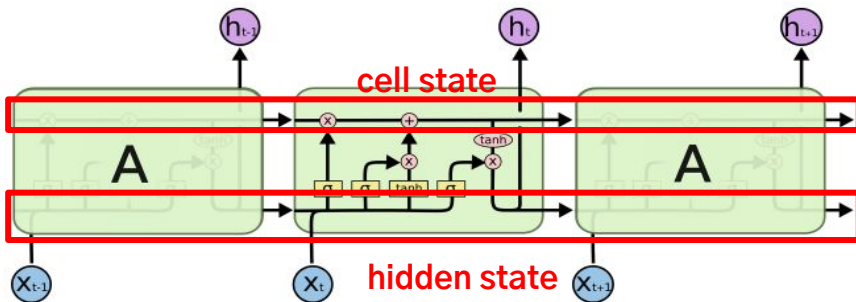
- Decoder 측 : 장난하냐.. 이 1장 보고 100장 내용을 어케알아!!

5. RNN 의 한계와 해결법 (LSTM, GRU)

5.2 LSTM

- Long Short Term Memory

- Vanilla RNN 의 단점인 장기 의존성 문제를 보완하여 나온 RNN 모델
- LSTM 은 **cell state** 에 핵심 정보들을 모두 담아두고, 각 타임 스텝 (time step)마다 필요한 정보만을 hidden state 에 업데이트하여 사용되도록 학습
- 오차역전파 (backpropagation) 진행시 RNN 은 **그레디언트 소실 문제 (Gradient Vanishing Problem)**가 있었지만, LSTM 은 이 문제를 **cell state 와 게이트 (gate)**라는 개념을 통해 **그레디언트 소실 문제를 개선함**

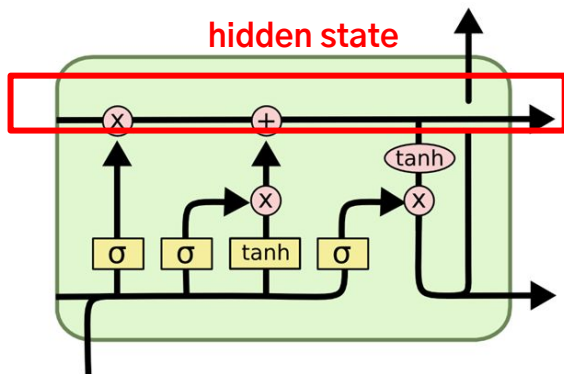


5. RNN 의 한계와 해결법 (LSTM, GRU)

5.3 GRU

- Gated Recurrent Unit

- LSTM 모델 구조를 더 단순화한 RNN 모델
- LSTM 은 2가지 상태 (state)와 3가지 게이트 (gate)를 활용한 반면,
GRU 는 1가지 상태 (state)와 2가지 게이트 (gate)를 활용함
- 다만 cell state 가 없이 하나의 state 만 사용하여, 장기 의존성 문제에는 LSTM 보다 취약함



T h a n k · Y o u